Listing 24.7 Creating an authorization policy with multiple requirements

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.services.AddAuthorization(options =>
{                                                          ❶
    options.AddPolicy(                                     ❶
        "CanEnterSecurity",                                ❶
        policyBuilder => policyBuilder                     ❶
            .RequireClaim(Claims.BoardingPassNumber));     ❶
    options.AddPolicy(                                     ❷
        "CanAccessLounge",                                 ❷
        policyBuilder => policyBuilder.AddRequirements(    ❸
            new MinimumAgeRequirement(18),                 ❸
            new AllowedInLoungeRequirement()               ❸
        ));
});
// Additional configuration
```

❶ Adds the previous simple policy for passing through security

❷ Adds a new policy for the airport lounge, called CanAccessLounge

❸ Adds an instance of each IAuthorizationRequirement object

You now have a policy called `"CanAccessLounge"` with two requirements, so you can apply it to a Razor Page or action method using the `[Authorize]` attribute, in exactly the same way you did for the `"CanEnterSecurity"` policy:

```
[Authorize("CanAccessLounge")]
public class AirportLoungeModel : PageModel
{
    public void OnGet() { }
}
```

When a request is routed to the AirportLounge.cshtml Razor Page, the authorize middleware executes the authorization policy and each of the requirements is inspected. But you saw earlier that the requirements are